

DOA aflevering 1

Exercise 1

a)

Main.cpp

```
1  #include <iostream>
2  #include "prng.h"
3  #include <vector>
4  #include <algorithm>
5
6  int main() {
7      int m = 0;
8      int n = 0;
9      int counter = 0;
10
11     std::cout << "Input the M amount: ";
12     std::cin >> m;
13
14     std::cout << "Input the N amount: ";
15     std::cin >> n;
16
17     std::vector<int> mvector;
18
19
20     while (m > 0) {
21         mvector.push_back(prng());
22         --m;
23     };
24
25     while (n > 0) {
26         int number = prng();
27
28         auto result = std::find(mvector.begin(), mvector.end(), number);
29         if (result != mvector.end()) {
30             ++counter;
31         }
32         --n;
33     }
34     for (auto i : mvector) {
35         std::cout << i << " ";
36     };
37
38     std::cout << std::endl;
39
40     std::cout << "N intergers was found " << counter << " amount of times in the array" << std::endl;
41
42     return 0;
43
44
45 }
```

PRNG.CPP

```
1  #include "prng.h"
2  #include <random>
3
4  int prng() {
5      std::random_device rd;
6      std::mt19937 gen(rd());
7      std::uniform_int_distribution<> dist(0, 100);
8
9      int m = dist(gen);
10
11
12     return m;
13 }
```

Prng.h

```
1  #ifndef PRNG_H
2  #define PRNG_H
3
4  int prng();
5
6  #endif
```

b)

for prng.cpp:

```
1  #include "prng.h"
2  #include <random>
3
4  int prng() {
5      std::random_device rd;           // O(1)
6      std::mt19937 gen(rd());          // O(1)
7      std::uniform_int_distribution<> dist(0,100); // O(1)
8
9      int m = dist(gen);               // O(1)
10
11     return m;
12 }
```

Hver linje er $O(1)$, fordi de alle udføres netop en gang uanset input. Dermed er funktionen `prng()` som helhed også $O(1)$.

For main.cpp:

```

1  #include <iostream>
2  #include "prng.h"
3  #include <vector>
4  #include <algorithm>
5
6  int main() {
7      int m = 0; // O(1)
8      int n = 0; // O(1)
9      int counter = 0; // O(1)
10
11     std::cout << "Input the M amount: "; // O(1)
12     std::cin >> m; // O(1)
13
14     std::cout << "Input the N amount: "; // O(1)
15     std::cin >> n; // O(1)
16
17     std::vector<int> mvector; // O(1)
18
19     while (m > 0) { // O(m)
20         mvector.push_back(prng()); // O(1) am
21         --m; // O(1)
22     } // loop: O(m)
23
24     while (n > 0) { // O(n)
25         int number = prng(); // O(1)
26
27         auto result = std::find(mvector.begin(), mvector.end(), number); // O(m)
28         if (result != mvector.end()) { // O(1)
29             ++counter; // O(1)
30         }
31         --n; // O(1)
32     } // loop: O(n*m)
33     for (auto i : mvector) { // O(m)
34         std::cout << i << " "; // O(1)
35     } // loop: O(m)
36
37     std::cout << std::endl; // O(1)
38
39     std::cout << "N intergers was found " << counter << " amount of times in the array" << std::endl; // O(1)
40
41     return 0;
42 }

```

Så har vi: $O(1) + O(m) + O(n \cdot m) + O(m)$. Det betyder dermed at programmets tidskompleksitet kan evalueres til: $O(n \cdot m)$.

Exercise 2

Hvis vi tager alle loops og regner på hvor effektive de er i forhold til Big O notation kan vi se at vi får udtrykket

$$N \cdot \log(N) \cdot N \cdot N + N^2$$

$$N^3 + N^2$$

Da N^3 er den mest tidskrævende forkorter vi til $O(N^3)$

$$O(N^3)$$

Exercise 3

(a):

1. $O(n)$

Første funktion har en kompleksitet på $O(n)$ fordi vi har et for loop der kører n gange og udfører en enkelt $++$ operation for hver iteration. Dermed $1 \cdot n = O(n)$

2. $O(n^2)$

Funktion nummer 2. er lidt samme scenarie som funktion 1, men hvor vi altså har et nested for loop inde i et for loop, og dermed $1 \cdot n \cdot n = O(n^2)$

3. $O(n^3)$

Funktion nummer 3. går lige skridtet videre fra funktion 2, i det at vi itererer indtil vi har nået $n \cdot n$ i det indre for loop. Derfor får vi $1 \cdot n \cdot (n \cdot n) = O(n^3)$

4. $O(n^2)$

Funktion nummer 4. er lidt anderledes i det at det indre loop afhænger af det ydre for loop. Ydre loop er helt som de andre funktioner og tæller op på i indtil n er nået. Det indre loop derimod stopper når tælleren j når i . Vi har altså $\frac{n \cdot (n-1)}{2}$

Det er $n \cdot (n - 1)$ fordi vi har et indre og ydre loop der hænger sammen, men ved $n = 0$ bliver en iteration af indre for loop sprunget over dermed $n-1$.

Derfor kan vi konkludere kompleksitet af $O(n^2)$ da vi forkorter udtrykket til at indeholde kun den mest væsentlige karakteristika, altså n^2

(b):

1.

$$n = 100$$

$$0,0002ms$$

$$n = 1000$$

$$0,0004ms$$

2.

$$n = 100$$

$$0,008ms$$

$$n = 1000$$

$$0,03153ms$$

3.

$$n = 100$$

$$0,3118ms$$

$$n = 1000$$

$$284,815ms$$

4.

$$n = 100$$

$$0,0024ms$$

$$n = 1000$$

$$0,1604ms$$

(c)

	1	2	3	4
n=100	0,0002ms	0,008ms	0,3118ms	0,0024ms
n=1000	0,0004ms	0,03153ms	284,815ms	0,1604ms

Som forventet ser vi at runtime på funktion 1. som vi via. Analyse har vurderet til at være en $O(N)$ funktion, er klart hurtigst. Og vi kan se at funktion 3. som vi har vurderet til at være $O(N^3)$, har klart den længste runtime og stiger eksponentielt i tid med højere sample size.

(d)

Funktion 3. Med -O compiling

$$n = 100$$

0,2042ms

$n = 1000$

181,965ms

Funktion 4. Med -O compiling

$n = 100$

0,0018ms

$n = 1000$

0,1068ms

Som vi kan se er cirka en tredjedel af runtime skåret af for 100 samples, og det samme gælder for vores forsøg med 1000 samples. Vi kan altså konkludere at -O til compiling hjælper væsentligt ifht. At skære ned på runtime

Exercise 4

Maxheap.h (pure virtual):

```
1  #pragma once
2
3  class MaxHeap {
4      public:
5          //checks if heap is empty
6          virtual bool isEmpty() const = 0;
7
8          //returns number of elements in the heap
9          virtual int size() = 0;
10
11         //adds an element to the heap
12         virtual void insert(const int x) = 0;
13
14         //finds the maximum element in the heap
15         virtual const int findMax() const = 0;
16
17         //deletes and returns the maximum element in the heap
18         virtual int deleteMax() = 0;
19     };
```

Vectorheap.h (arvning til Maxheap.h)

```
1  #include "Maxheap.h"
2  #include <vector>
3
4  class Vectorheap : public MaxHeap {
5      public:
6          bool isEmpty() const override;
7          int size() override;
8          void insert(const int x) override;
9          const int findMax() const override;
10         int deleteMax() override;
11
12     private:
13         std::vector<int> heap;
14 };
```

Vectorheap.cpp (implementering)

```

1  #include "Maxheap.h"
2  #include <stdexcept>
3
4  bool Maxheap::isEmpty() const {
5      return heap.empty();
6  }
7
8  int Maxheap::size() {
9      return heap.size();
10 }
11
12 void Maxheap::insert(const int x) {
13     heap.emplace_back(x);
14 }
15
16 const int Maxheap::findMax() const {
17     if(isEmpty()) {
18         throw std::runtime_error("Empty heap")
19     }
20
21     auto it = heap.begin();
22     int MAX = *it;
23
24     for(; it != heap.end(); ++it) {
25         if(*it > MAX) {
26             MAX = *it;
27         }
28     }
29     return MAX;
30 }
31
32 int Maxheap::deleteMax() {
33     int MAX = findMax();
34     auto it = find(heap.begin(), heap.end(), MAX);
35     heap.erase(it);
36     return MAX;
37 }

```

1. isEmpty(): $O(1)$, da .empty() tjekker en enkelt variabel for om vektoren er tom.
2. size(): $O(1)$, da .size() tjekker en enkelt variabel for længden af vektoren.
3. insert(const int x): amortiseret konstant. .emplace_back() appender et element, men når arrayet er fuldt, kopieres data over i et nyt, dobbelt så stort array, så de fleste insert()s tager $O(1)$ tid, men nogle gange en del længere tid.
4. findMax(): $O(n)$, da hele arrayet gennemses hver gang.
5. deleteMax(): $O(n)$, da hele arrayet gennemses en gang for at finde max-værdien med findMax() ($O(n)$) og .erase() sletter en værdi ($O(1)$) og derefter shifter

halen for at udfylde dette hul i vektoren, som i værste tilfælde shifter $n-1$ elementer ($O(n)$).

Exercise 5

(a)

Se vedhæftet kode

(b)

```
// BOOK
Book::Book(int bookId, int bookCategory)
|   : id(bookId), category(bookCategory), borrowed(false) {
};

int Book::getId() const {
|   return id;
};

int Book::getbookcategory() const {
|   return category;
};

bool Book::getborrowedstate() const {
|   return borrowed;
};

void Book::setReturn() {
|   borrowed = false;
}

void Book::setBorrow() {
|   borrowed = true;
}
```

Først og fremmest valgte vi at lave en book klasse som selvfølgelig har ID, Kategori og om bogen er lånt ud. Borrowed er selvfølgelig sat til at være false når en ny bog bliver skabt. Udover det har vi setters og getters som forventet.

```
std::vector<int> Library::countCategories() {  
    std::vector<int> AllCategories;  
  
    for (const Book& b : books)  
    {  
        AllCategories.push_back(b.getbookcategory());  
    }  
  
    return AllCategories;  
}
```

countCategories() anvender en vector af integers. Vector er anvendt fordi vi ikke har et behov for at gå frem og tilbage i containeren, som feks. Med en liste. Yderligere er der ikke behov for at kunne ændre på rækkefølgen så at et element kan blive tilføjet feks. I midten af containeren. Derfor er vector en tilstrækkelig container.

```

void Library::addBook(int bookid, int category) {

    //ny bog laves
    Book newbook(bookid, category);
    books.push_back(newbook);

    std::vector<int> AllCategories = countCategories();
    int countOfCategory =0;
    int categories =1;

    //tæller den nyindsatte kategori
    for (int i =0; i<AllCategories.size(); i++){
        if (category == AllCategories[i])
        {
            ++countOfCategory;
        }
    }

    //tæller mængden af kategorier
    for (int i =0; i<AllCategories.size(); i++){
        if (AllCategories[i] != category)
        {
            ++categories;
        }
    }

    //finder gennemsnit blandt alle kategorier
    int avg = AllCategories.size() / categories;

    //hvis kategori indsat er mere end dobbelt så stor som gennemsnit, send besked.
    if (countOfCategory > avg*2)
    {
        std::cout<<"category " <<category<< " exceeds limit"<<std::endl;
    }
}

```

I vores addBook() funktion anvendes samme vector som i countCategories(). Heldigvis tillader vector .size() funktionen som kan returnere mængden af elementer i den givne vector. Endnu et pluds for vores valgte container.

(c)

```
std::vector<int> Library::countCategories() {  
    std::vector<int> AllCategories;  
  
    for (const Book& b : books)  
    {  
        AllCategories.push_back(b.getbookcategory());  
    }  
  
    return AllCategories;  
}
```

countCategories() funktionen har en kompleksitet af $O(n)$ fordi der loopes gennem n mængde af bøger, og skubber dem ind i vores vector. Den resterende return operation er ligegyldig, fordi den kun forekommer en enkelt gang pr. Eksekvering af funktion.

```

void Library::addBook(int bookid, int category) {

    //ny bog laves
    Book newbook(bookid, category);
    books.push_back(newbook);

    std::vector<int> AllCategories = countCategories();
    int countOfCategory =0;
    int categories =1;

    //tæller den nyindsatte kategori
    for (int i =0; i<AllCategories.size(); i++){
        if (category == AllCategories[i])
        {
            ++countOfCategory;
        }
    }

    //tæller mængden af kategorier
    for (int i =0; i<AllCategories.size(); i++){
        if (AllCategories[i] != category)
        {
            categories;
        }
    }

    //finder gennemsnit blandt alle kategorier
    int avg = AllCategories.size() / categories;

    //hvis kategori indsat er mere end dobbelt så stor som gennemsnit, send besked.
    if (countOfCategory > avg*2)
    {
        std::cout<<"category " <<category<< " exceeds limit"<<std::endl;
    }
}

```

addBook kompleksitet er $O(n)$ fordi der er samtlige for loops med kompleksitet $O(n)$, men de er aldrig nested på en måde hvorpå de gror eksponentielt i kompleksitet. De resterende operationer er relativt trivielle i deres kompleksitet.

```

void Library::displayAllBooks() {
    for (const Book& b : books) {
        std::cout << "BookID: " << b.getid()
            << " BookCategory: " << b.getbookcategory()
            << " Borrowedstate: " << (b.getborrowedstate()) ? "Borrowed" : "not borrowed") << std::endl;
    }
}

```

displayAllBooks kompleksitet er $O(n)$ fordi vi itererer over længden af books, men der sker kun en enkelt std::cout operation inde i loopet.

```

void Library::displayAvailableBooks() {
    for (const Book& b : books) {
        if (b.getborrowedstate() == false) {
            std::cout << "BookID: " << b.getid()
                << " BookCategory: " << b.getbookcategory()
                << " Borrowedstate: " << (b.getborrowedstate() ? "Borrowed" : "not borrowed") << std::endl;
        }
    }
}

```

displayAvailableBooks kompleksitet er $O(n)$. vi har et if statement inde i for loopet. Det er dog ikke nok til at kunne klacifcere funktionen som andet end $O(n)$ da et if statement ikke er nok til at få funktionen til at vokse i kompleksitet eksponentielt.

```

✓ bool Library::borrowBook(int a) {
✓     for (Book& b : books) {
✓         if (b.getid() == a) {
✓             if (b.getborrowedstate() == false) {
✓                 b.setBorrow();
✓                 return true;
✓             }
✓             else {
✓                 return false;
✓             }
✓         }
✓     }
✓     return false;
✓ }

```

borrowBook kompleksitet er $O(n)$ da vi har et for loop der looper igennem n books, mens at der bliver lavet to if operationer inde i loopet. Det nastede if statement gør at vi i værste tilfælde foretager os $n \cdot 3$ operationer for alle bøger. Da det ikke er nok til at vi kan kalde det $O(n^2)$ eller $O(n \log n)$ kalder vi det $O(n)$ af kompleksitet

```

void Library::displayBorrowedBooks() {
    for (const Book& b : books) {
        if (b.getborrowedstate()) {
            std::cout << "BookID: " << b.getid()
                << " BookCategory: " << b.getbookcategory()
                << " Borrowedstate: " << (b.getborrowedstate() ? "Borrowed" : "notborrowed") << std::endl;
        }
    }
}

```

displayBorrowedBooks kompleksitet er $O(n)$ da indholdet er af samme kompleksitet som displayAllBooks

```

bool Library::returnBook(int a) {
    for (Book& b : books) {
        if (b.getid() == a) {
            if (b.getborrowedstate() == true) {
                b.setReturn();
                return true;
            }
            else {
                return false;
            }
        }
    }
    return false;
}

```

returnBook kompleksitet er $O(n)$ af samme grund som borrowBook

```

int Library::countBooksInCategory(int a) {
    int countbooks = 0;
    for (const Book& b : books) {
        if (b.getbookcategory() == a) {
            ++countbooks;
        }
    }
    return countbooks;
}

```

Int countbooks tælder som 1 operation. Derudover har vi et for loop der itererer gennem alle books, og checker for bogens kategori med et if statement, og tæller countbooks op

hvis kategorien er a. Til at slutte af med har vi en sidste operation som er returnering af vores optælning countbooks. Dermed $1 + O(n) \cdot 3 + 1$

Dermed kan vi konkludere at kompleksiteten er $O(n)$